

Otimização em Engenharia de Software

Por que, quando e como aplicar?

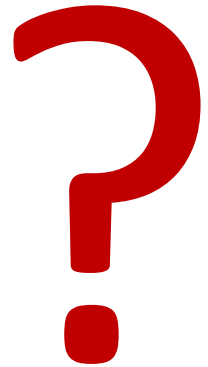
Thelma Elita Colanzi

Universidade Estadual de Maringá (UEM)

thelma@din.uem.br

Otimização em Engenharia de Software

Por que e quando aplicar.



Por que e Quando ?

Problemas Complexos

- 1:** Grande Espaço de Busca
- 2:** Ausência de Solução Ótima Conhecida

Exemplos de Problemas Complexos em Engenharia de Software

Planejamento de Releases

Estimativa de Custos

Alocação de Pessoal

Exemplos de Problemas Complexos em Engenharia de Software

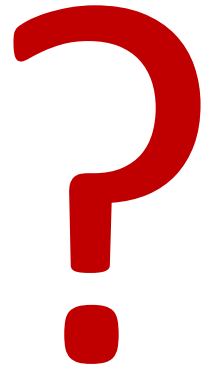
**Otimização de Projeto de
Arquitetura de Software**

Otimização de Código Fonte

**Geração, Seleção e Priorização de
Casos de Teste**

Otimização em Engenharia de Software

Como aplicar.



Como aplicar?

Escolha de uma **Representação** para as soluções do problema

Definição da **Função(ões) de Avaliação** e das **Restrições** do problema

Escolha e Aplicação de **Algoritmos de Busca**

A large teal circle is positioned on the right side of the slide, partially overlapping the white background. The title text is centered within this circle.

Projeto de Arquitetura de Software

Projeto de Arquitetura

Representa a ligação (crítica) entre requisitos e implementação.
(Garlan, 2014)

Sua importância tem crescido significativamente devido à demanda de desenvolvimento de **software de alta complexidade** e requisitos de qualidade desafiadores.

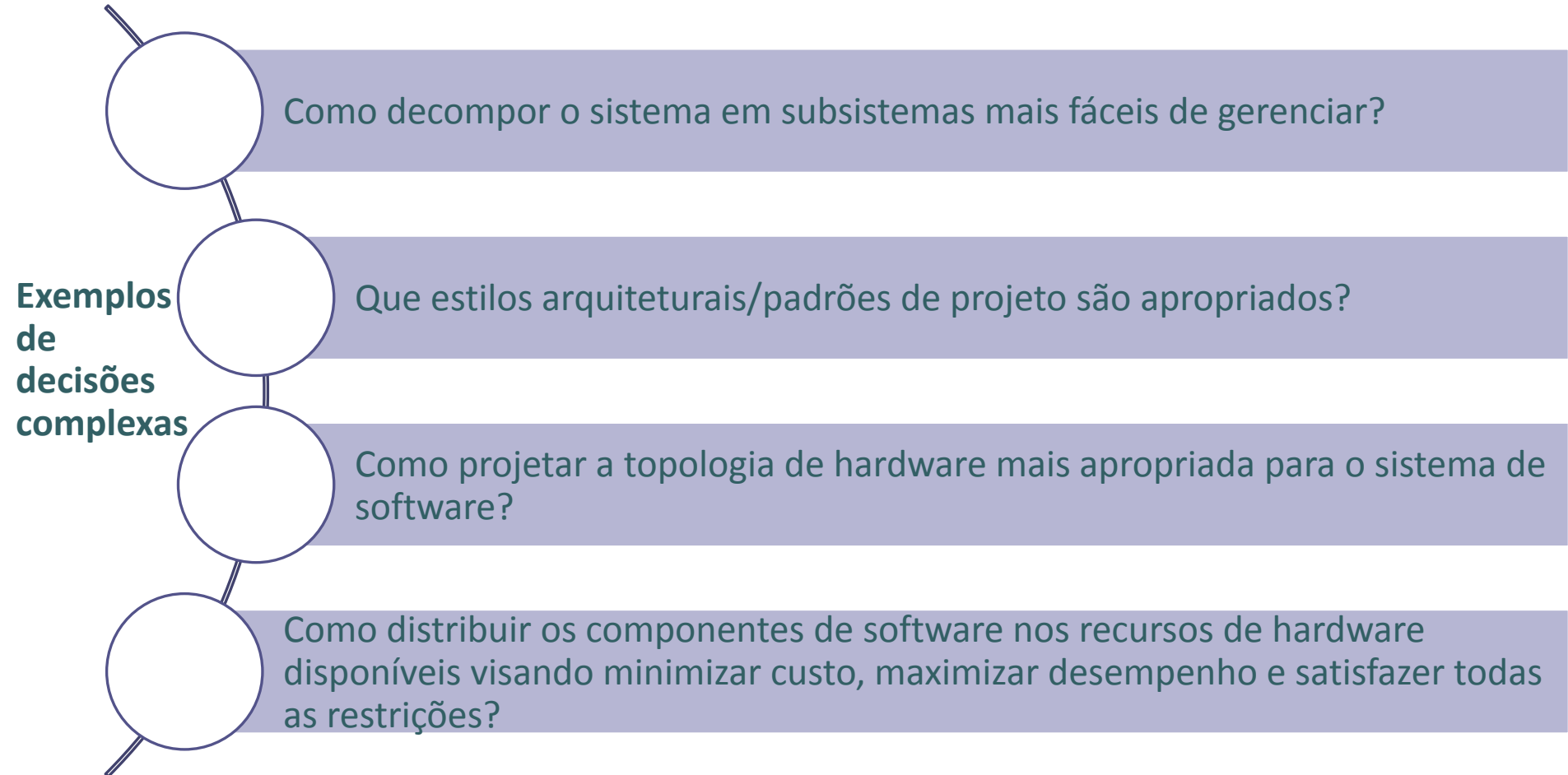
Software Embutido

Software Concorrente

Arquitetura em larga escala:

- Sistemas que incluem outros sistemas/componentes
- Linha de Produto de Software
- Sistemas de Sistemas

Projeto de Arquitetura



**Problema
Multi-objetivo**

Projeto de Arquitetura de Software é uma atividade que demanda grande esforço dos arquitetos de software.

Vários fatores (propriedades arquiteturais, requisitos de qualidade) precisam ser priorizados ao mesmo tempo.

Os fatores que influenciam o projeto muitas vezes são conflitantes.

Esses fatores têm diferentes prioridades de acordo com as restrições do domínio ou da empresa.

Várias alternativas de projeto costumam ser válidas para um mesmo software.

Arquitetura em Larga Escala: Linha de Produto de Software (LPS)



Explorar as semelhanças entre os produtos para aumentar o reúso de artefatos.

Linha de Produto de Software (LPS)



Product Line Architecture - PLA

Um dos principais artefatos que viabilizam o reuso em uma LPS é a arquitetura da LPS (*Product Line Architecture - PLA*).

Uma PLA inclui componentes que realizam características comuns e variáveis entre os produtos da LPS.

- Visão lógica da arquitetura - abstrações fundamentais em termos de classes e seus relacionamentos

Otimização de Projeto de PLA

Principal motivação

Reconhecer um bom projeto é mais fácil do que obtê-lo!

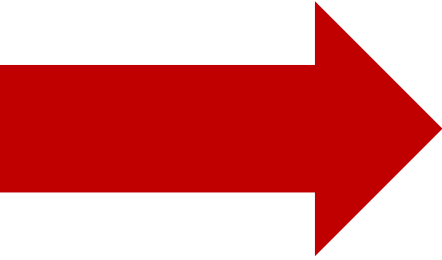
Problema

Encontrar um conjunto de alternativas de projeto de arquitetura de LPS que otimize as **propriedades arquiteturais priorizadas** pelo arquiteto.

Maximizar Propriedades da PLA

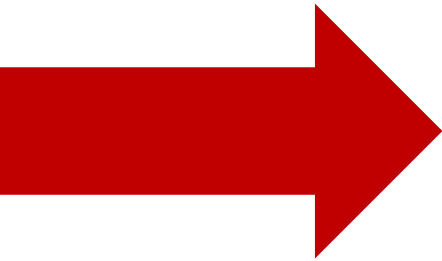
Projeto de Arquitetura de LPS

Projeto de Arquitetura de LPS (PLA)



Diminuir custos e time-to-Market

Maximizar reusabilidade



Facilitar evolução e manutenção da LPS

Maximizar extensibilidade

Maximizar modularidade

Otimização de projeto de PLA – visão lógica

Otimização de propriedades
arquiteturais do projeto da
PLA

Algoritmos: GA, NSGA-II, PAES,
Alg. meméticos, NSGA-III

Tratamento
multi-objetivo

Métricas convencionais de
projeto OO e métricas de LPS

Desafios

1

Representação do problema

2

Modelo de avaliação

3

Operadores de Busca específicos para a representação adotada

4

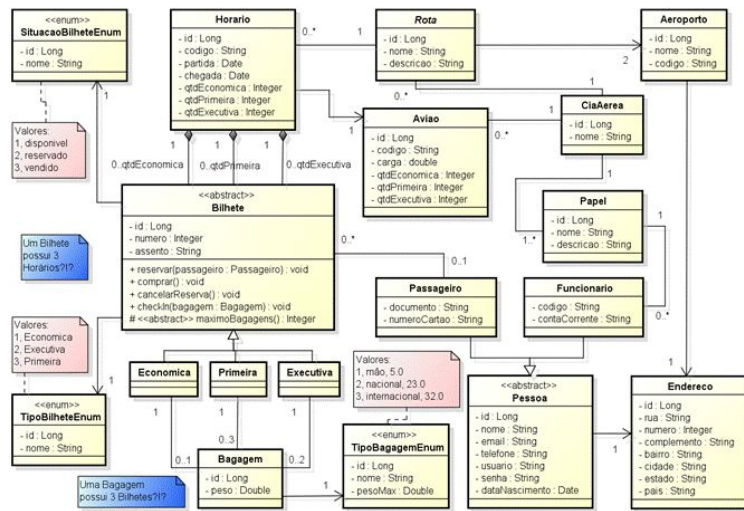
Algoritmo de busca mais apropriado

Desafios

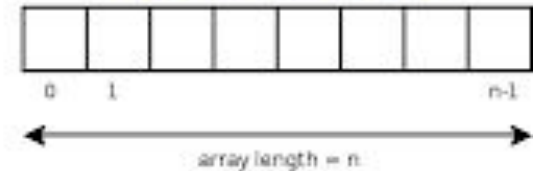


1

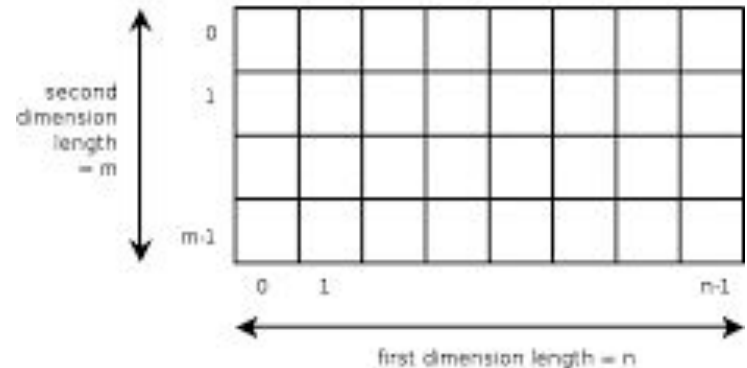
Representação do problema



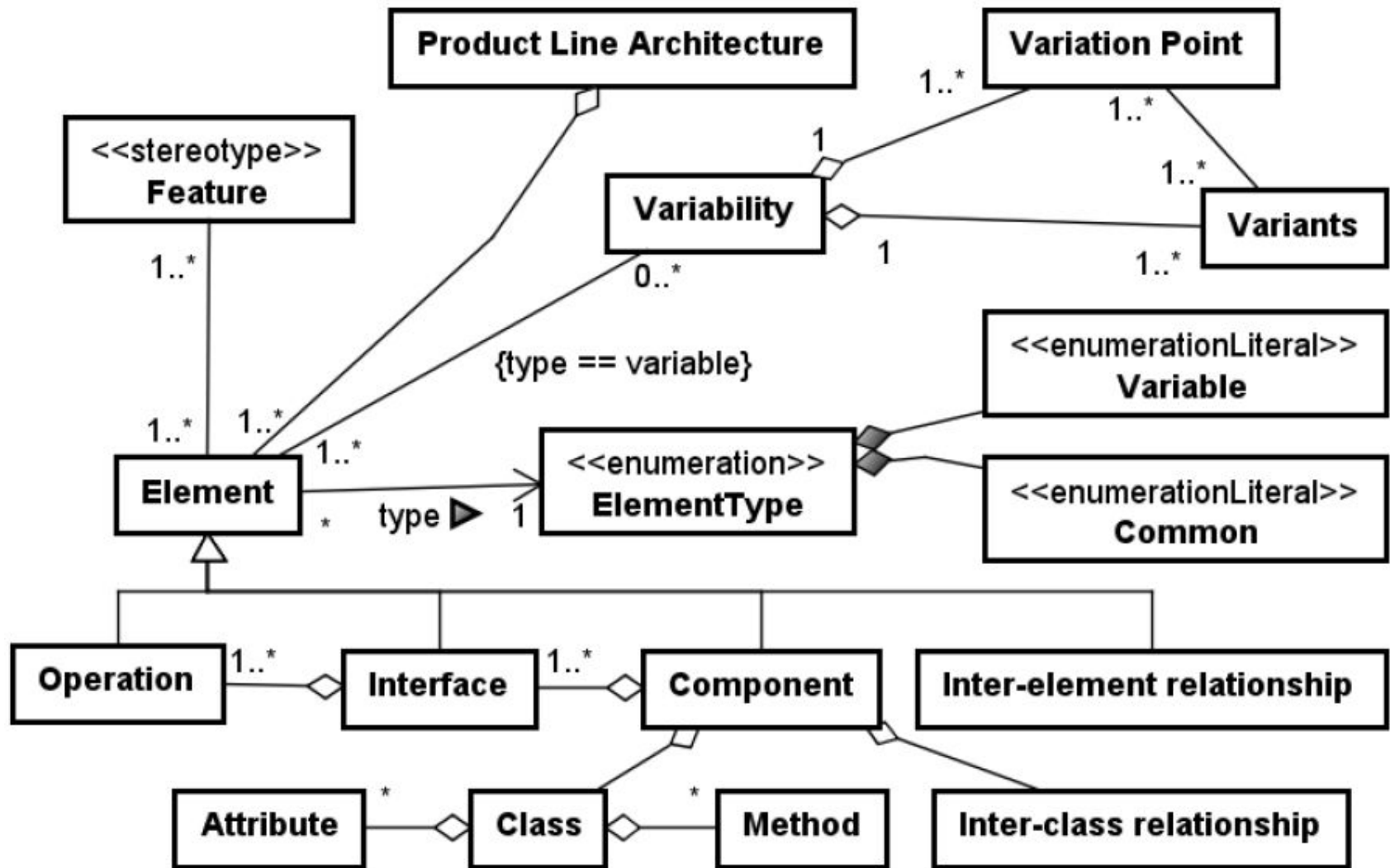
One-dimensional array



Two-dimensional array



Metamodelo da PLA



Desafios

2

Modelo de avaliação

- Encontrar métricas propostas na literatura
- Definição de 17 funções objetivo que medem propriedades como:

Métricas dirigidas a modularização de features

Métricas convencionais (acoplamento, coesão, tamanho, etc.)

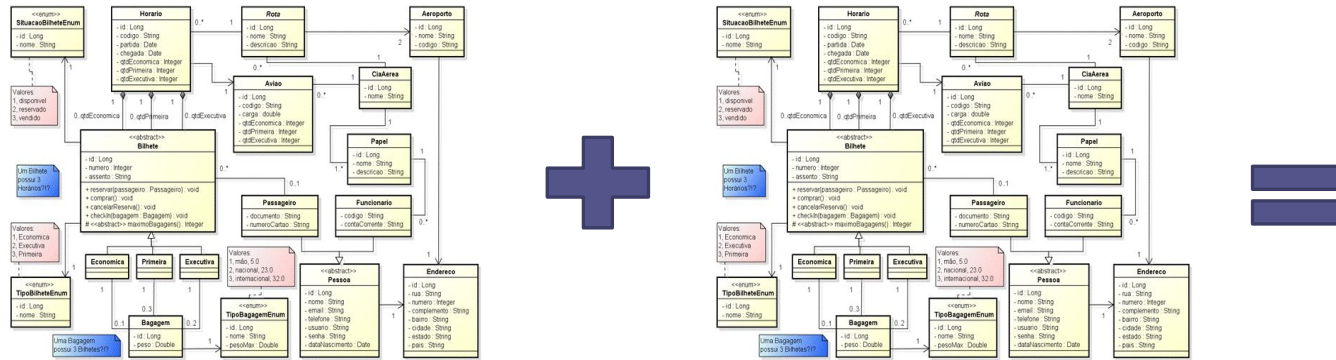
Extensibilidade do projeto de arquitetura da LPS

Variabilidade/Similaridade da PLA

Desafios

3

Operadores de Busca específicos para a representação adotada



8 operadores de mutação e 3 operadores de cruzamento específicos

Operadores de Busca para Projeto de PLA

Operadores de Mutação Convencionais:

- Mover métodos/atributos/relacionamentos;
- Adicionar classes
- Adicionar pacotes e interfaces

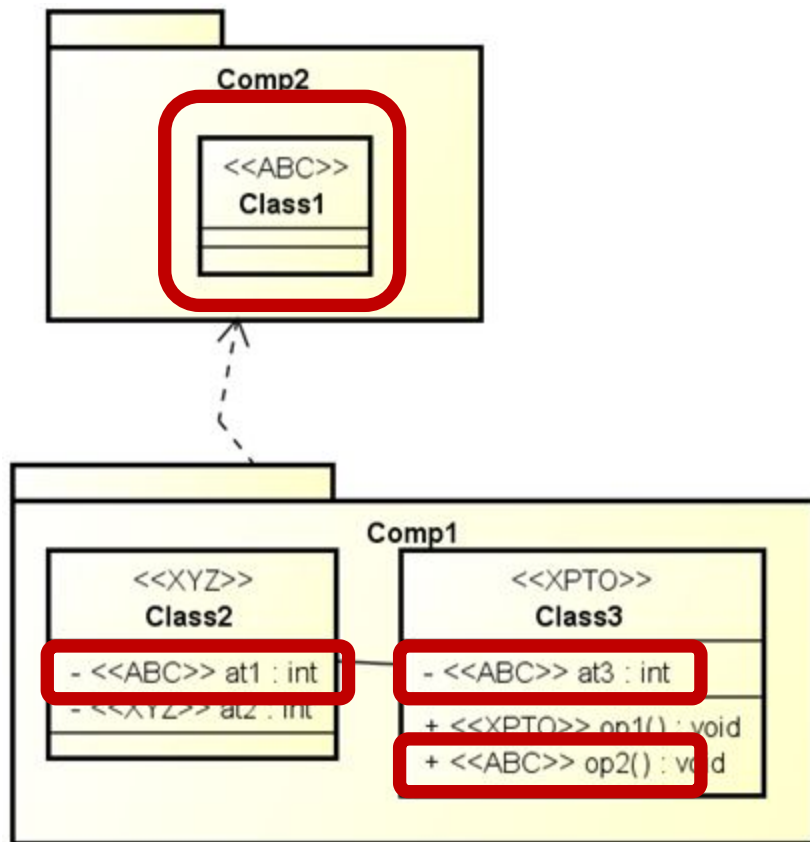
Operadores de Mutação dirigido a *Features*

Operador de Mutação para refatorar projeto utilizando padrões de projeto GoF (Strategy, Bridge e Mediator)

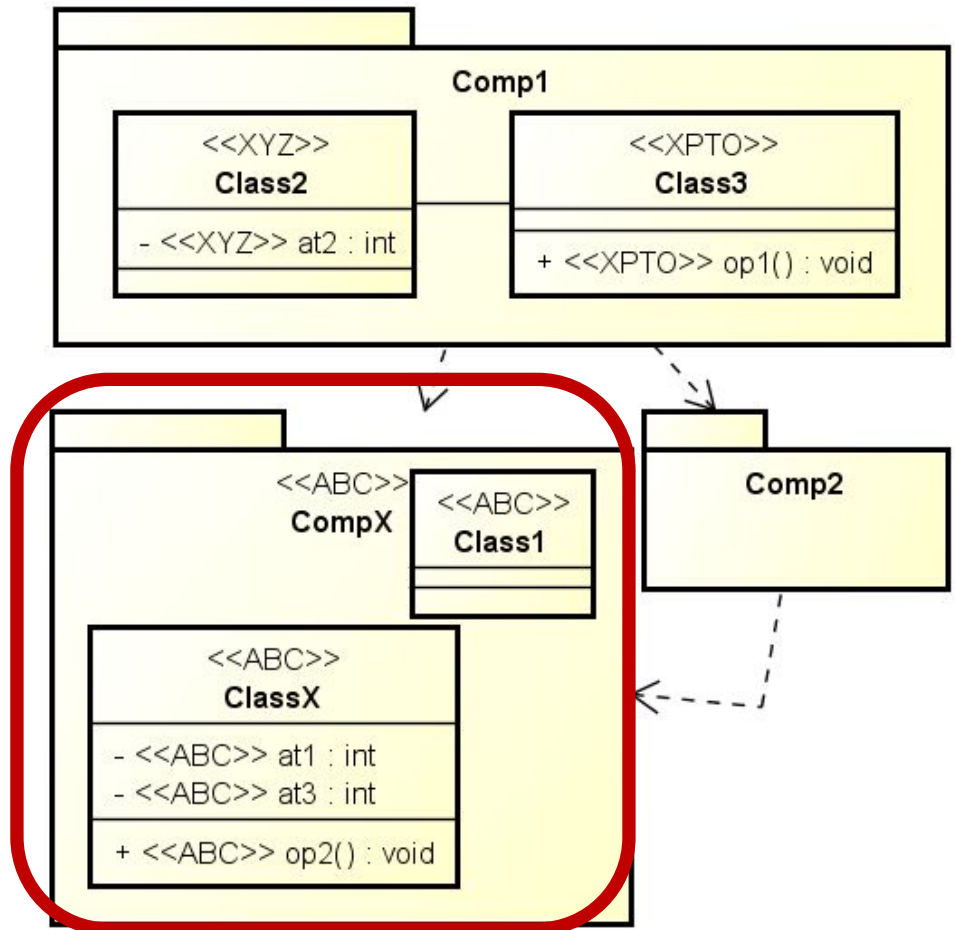
Operadores de Cruzamento dirigido a *Features*

Feature-driven Mutation

Antes



Depois



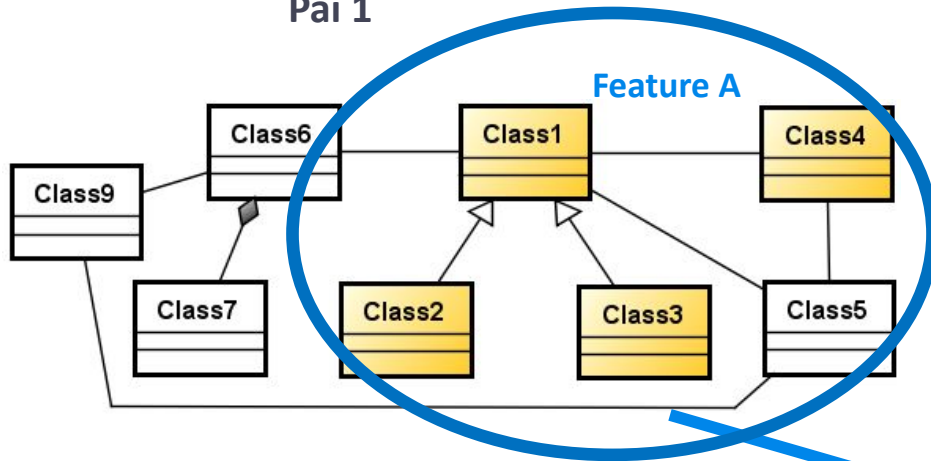
Feature-driven Crossover

Antes

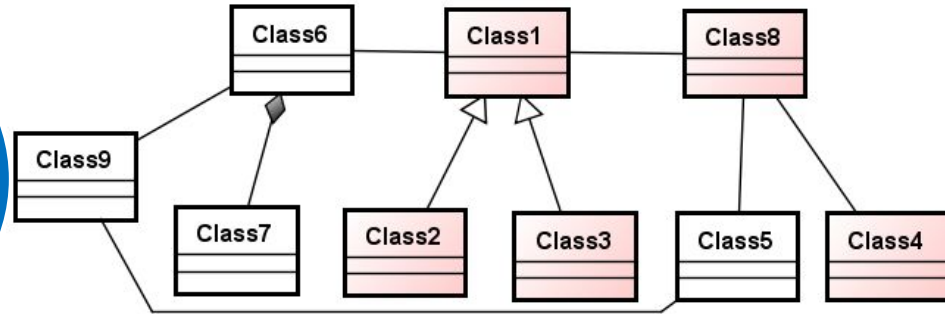
Depois

Pai 1

Feature A

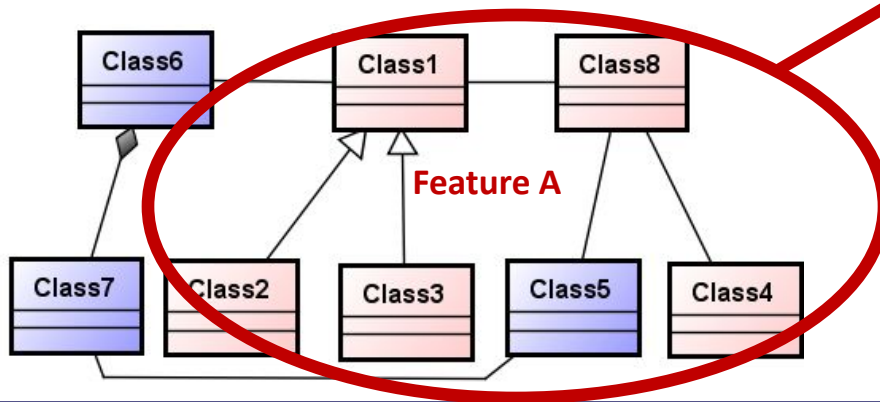


Filho 1

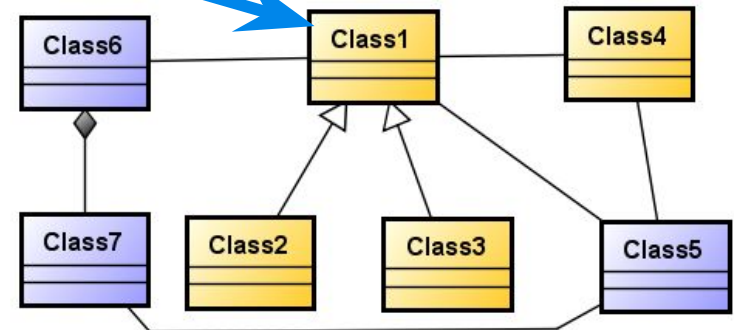


Pai 2

Feature A



Filho 2



Desafios

4

Algoritmo de busca mais apropriado

Avaliação de **algoritmos genéticos, NSGA-II, PAES e algoritmos meméticos** utilizando algumas arquiteturas de LPS de diferentes domínios e portes, buscando otimizar 2 e 3 propriedades arquiteturais.

NSGA-II e algoritmo memético obtiveram os melhores resultados nos estudos experimentais realizados.

Alguns resultados



Soluções com melhor modularização de *features*

Algoritmo memético obtém melhores soluções que o NSGA-II puro quando a PLA original tem melhor modularização de *features*

Correlações entre funções objetivo foram identificadas em estudos preliminares permitindo a derivação de diretrizes para a seleção de funções objetivo

Diretrizes para prevenção de anomalias arquiteturais em projetos obtidos automaticamente

Avaliação qualitativa por especialistas aponta resultados positivos e direções de melhorias para a MOA4PLA

Settings

Select the Optimization Algorithm

NSGAII

Number of runs

1

Number of fitness evaluations

300

Population Size

20

Objective Functions

At least one objective function is **required**

☐ ACLASS

☐ ACOMP

☐ TV

☐ RCC

☐ COE

☐ CM

☐ DC

☐ EC

☐ ELEG

☐ FM

☐ LCC

☐ EXT

☐ SD

☐ SV

☐ TAM

☐ WOCSCCLASS

☐ CS

☐ LFCC

☐ FDAC

☐ CIBF

Interaction - Learning Settings

☐ Interact during the optimization

Select the clustering algorithm

KMEANS

Select the clustering moment

INTERACTIVE

Max interactions

0

First interaction

0

Interval

0

Crossover Operators

☐ Crossover rate 0  1

☐ PLA Feature-Driven Crossover

☐ PLA Complementary Crossover

☐ PLA Modular Crossover

Mutation Operators

☒ Mutation rate 0  1

☒ Feature-driven Operator

☒ Move Method Mutation

☒ Move Attribute Mutation

☒ Move Operation Mutation

☒ Add Class Mutation

☒ Add Manager Class Mutation

☒ Feature-driven Operator For Class

Paths

Select the builder

SMARTY

Select the input PLA

agm1.smt



OPLA -Tool v2.0



Execution Configuration



Design Patterns



Experiments



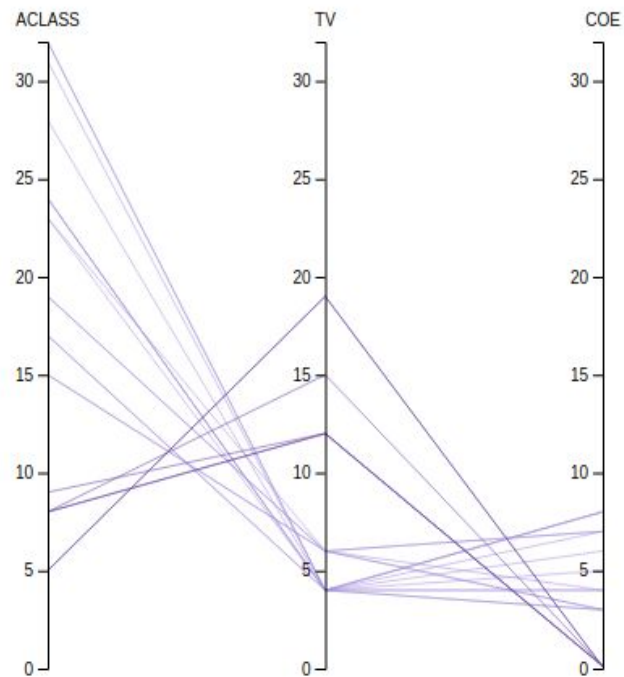
Results Analysis

Select the experiments...

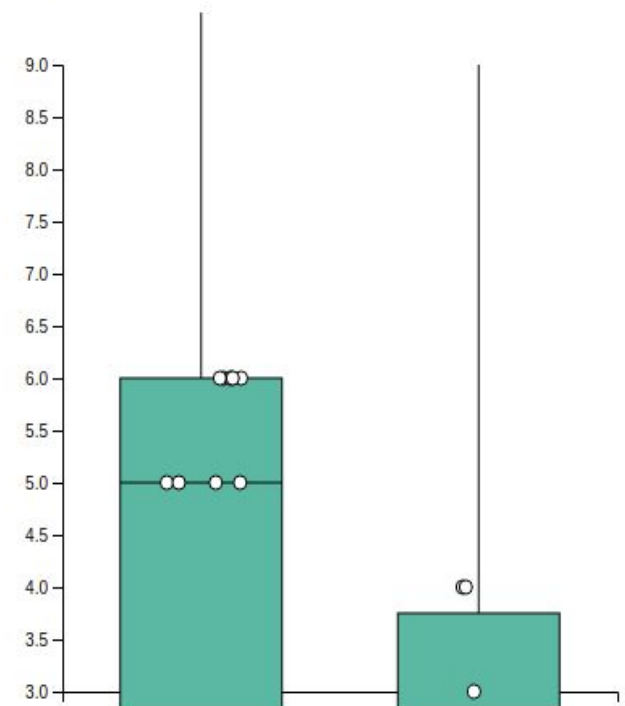
3 - agm1. - 2020-06-09 02:39:06 ✕

1 - agm1. - 2020-06-08 08:10:50 ✕

Parallel Coordinates



Boxplot



OPLA -Tool v2.0

Oportunidades de Pesquisa

Oportunidades de Pesquisa

Novos Operadores de Busca

Proposta de novos operadores de busca específicos para o problema ou para o domínio.

Novas Abordagens

Aplicações de outros algoritmos de busca em problemas existentes. Estudos comparativos de técnicas de busca.

Hibridização

Combinação de métodos na geração de abordagens mais eficientes.

Oportunidades de Pesquisa

Análise de Sensibilidade

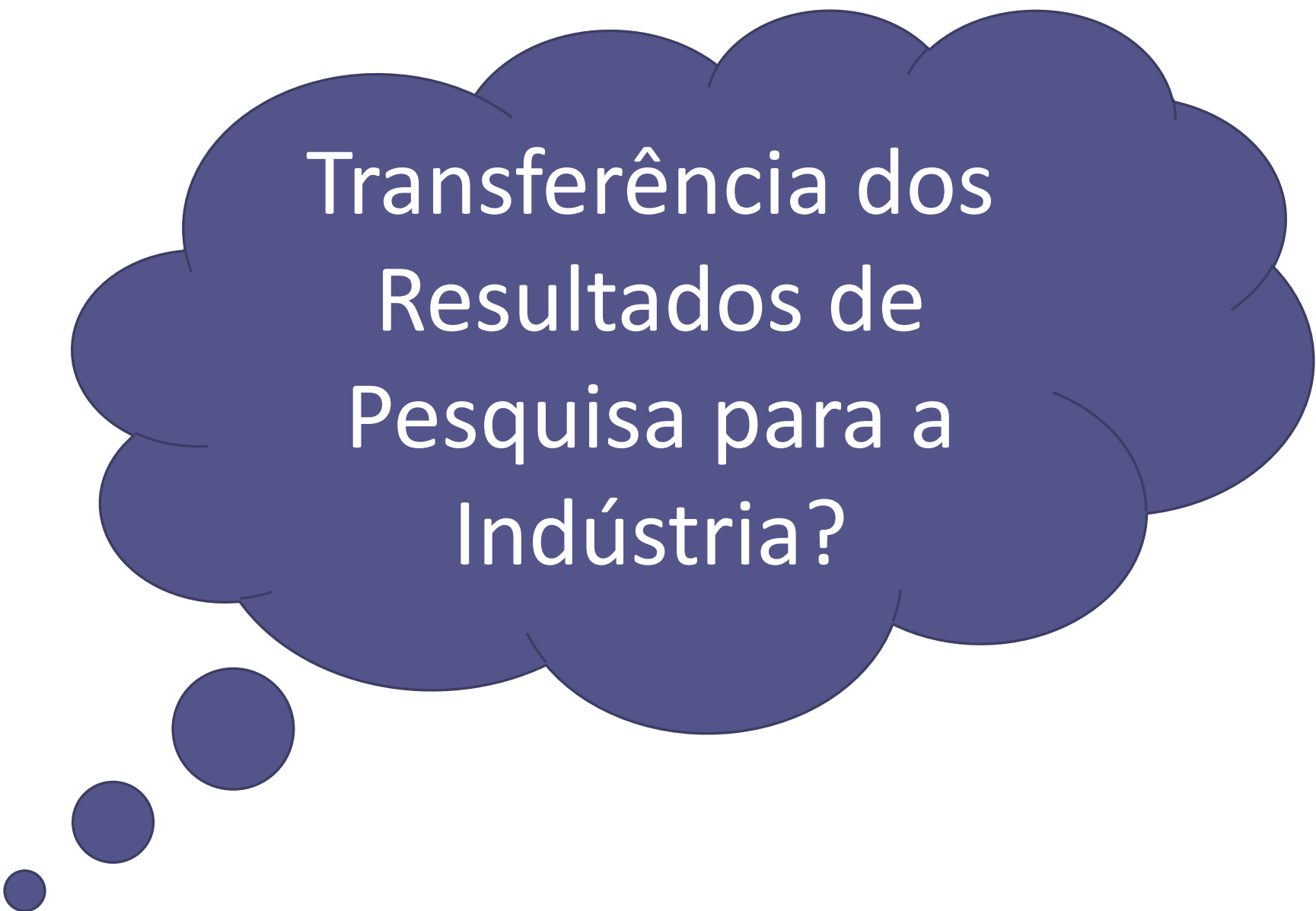
Avaliação da influência de variáveis na qualidade de resultados gerados.

Otimização Interativa

Estratégias de incorporação do julgamento humano durante o processo de otimização.

Competitividade Humana

Análise da competitividade dos resultados gerados automaticamente em comparação dos resultados produzidos por humanos.



Transferência dos
Resultados de
Pesquisa para a
Indústria?



Sapienz



Referências

Colanzi, T. E. Uma abordagem de otimização multiobjetivo para projeto arquitetural de linha de produto de software. Tese de Doutorado, Universidade Federal do Paraná, Curitiba, 2014

T. E. Colanzi, W. K. G. Assunção, S. R. Vergilio, P. R. Farah, G. Guizzo: The Symposium on Search-Based Software Engineering: Past, Present and Future. Inf. Softw. Technol. 127: 106372, 2020.

K. Deb, A. Pratap, S. Agarwal, e T. Meyarivan. A fast and elitist multiobjective genetic algorithm: NSGA-II. IEEE Transactions on Evolutionary Computation, 6(2):182 {197, abril de 2002.

Goldberg, D. Genetic Algorithms in Search, Optimization and Machine Learning. Addison-Wesley Longman Publishing Co., 1989.

HARMAN, M.; MANSOURI, S. A.; ZHANG, Y. Search based software engineering: A comprehensive analysis and review of trends techniques and applications. Department of Computer Science, King's College London, Tech. Rep. TR-09-03, 2009.

HARMAN, M.; MANSOURI, A.; ZHANG, Y. Search Based Software Engineering: Trends, Techniques and Applications. ACM Computing Surveys. 45(1): Article 11, 2012.

HARMAN, M.; MCMINN, P.; SOUZA, J. T.; YOO, S. Search Based Software Engineering: Techniques, Taxonomy, Tutorial. Lecture Notes in Computer Science, v. 7007, p. 1-59, 2012.

Sapienz: Facebook's push to automate software testing. VentureBeat. 30 December 2018.
<https://venturebeat.com/2018/12/30/sapienz-facebooks-push-to-automate-software-testing/>

Obrigada!

Thelma Elita Colanzi

Universidade Estadual de Maringá (UEM)

thelma@din.uem.br

